

Final Assessment-Alternative Assessment

Semester	202520	Division	CIS
df = pd.read_csv("daily_food_delivery_orders.csv") df	Final_Project_Portfolio	Program	Bachelor of Information Technology/ Bachelor of Information Systems

Course Code	CDS 2413		
Course Title	INTRODUCTION TO DATA SCIENCE		
Course Instructor		CRN	
Assessment	Project Deliverable 2 – Machine Learning Model Report	Submission Date	Week 13

Student Declaration:

Academic Integrity Statement

In accordance with the HCT Academic Integrity Policy

- Students are required to refrain from all forms of academic integrity breaches as defined and explained by HCT.
- A student found guilty of having committed acts of academic integrity breach(es) will be subject to the relevant sanctions as outlined by HCT.

إفادة النزاهة الأكاديمية

وفقاً لسياسة كليات التقنية العليا للنزاهة الأكاديمية

- على الطلبة الالتزام بلوائح وقواعد النزاهة الأكاديمية، كما هو مبين وموضح في السياسات والإجراءات الخاصة بكليات التقنية العليا.
- في حالة ارتكاب الطالب أي شكل من أشكال الإخلال بالنزاهة الأكاديمية، سيتعرض إلى العقوبات الموضحة في السياسات ذات الصلة.

This assignment is entirely my/our own work except where I have duly acknowledged other sources in the text and listed those sources at the end of the assignment. I have not previously submitted this work to the HCT, or any other entity. I understand that I may be orally examined on my submission.

Student HCT ID(s):	H00496463	H00542955	H00532391	H00498422
Student Name(s):	Mariam Alkatheeri	Fatima Majed	Jawaher Alameri	Ayesha Almemari
Student Signature				

Chatbot (e.g. ChatGPT) statement

This submission allows you to use AI generated tools, but the prompt used must be stated and the response must be in red font. Include References to ChatGPT.

Instructions

Remove instructions and hints from the template before submitting the final report to BBLearn.

CLO3: Machine Learning Model (25 Points)

Preprocessing(5 Points)

Data Collection(1 Point)

Python Libraries: The main libraries used in reading the data are pandas, which is used when handling CSV/Excel files, and numpy, which is used in carrying out mathematical operations.

Variable Selection: The process through which you can select and segregate variables from the data set is by indexing the DataFrame.

- **Independent Variables (X):** These include the features being used to carry out the predictions (customer_age, delivery_distance_km, restaurant_type). They are usually selected through `df.iloc[:, :-1].values`.
- **Dependent Variable (Y):** This is the variable that we wish to predict. It is typically selected through `df.iloc[:, -1].values`.

Data Cleaning (1 Point)

The Approach: In order not to train your model based on "noise," we must preprocess the raw data first. Imagine data preprocessing as the crucial preparation stage—just like preparing your ingredients before making a five-star dish. We aim at neutralizing any possible issues that may ruin our forecasts. With the elimination of missing values and normalization, we make sure to have a robust base for further algorithm training.

Important Examples:

- **Mean Imputation:** Instead of dropping any records due to missing values, we substitute those with averages in numerical columns (customer_age, delivery_partner_rating). That way, we keep the number of samples while preserving the statistical properties of our data.
- **Duplicates Elimination:** We deliberately delete all duplicates. This way, we protect our model from focusing excessively on those values, which would lead us to biases in our final results.

Treating the Categorical Variables (1 Point)

Types Present:

- **Nominal:** Restaurant_type (bakeries, Italian restaurants, cafes, etc.) and payment_method (UPI, card, cash) since there is no intrinsic ordering within them.
- **Binary:** Order_status when categorized into “Delivered” versus “Not delivered”.
- **Methods of Encoding:**

- **One-Hot Encoding:** Employed to transform nominal independent variables (restaurant_type) into several binary variables to avoid the model considering any numerical ranking between categories.
- **Label Encoding:** Employed for the dependent variable (the target) to encode the textual value into numerical (for instance, 0 and 1).

Splitting the Dataset (1 Point)

- **Significance:** Data splitting enables model evaluation on unseen data and aids in identifying overfitting problems as well as assessing the generalizability of the model.
- **Split Ratio:** The ratio usually employed is an 80/20 split. This means that 80 percent of the data is utilized for training purposes, and 20 percent is reserved for testing purposes.

Featuring the Dataset (1 Point)

Feature Scaling: Feature scaling is performed on the data set.

Feature Scaling Method: Standardization (**StandardScaler**) is employed to transform numerical features (such as order_value and customer_age) such that their distribution has a zero mean and unit variance.

Rationale: Feature scaling is essential since different features are expressed in various units. For example, age and order value can be expressed in years and dollars, respectively. Failure to perform feature scaling will result in numerical variables having large ranges dominating the learning process.

Regression Model (7 Points)

Building the Regression Model (6 points)

The daily_food_delivery_orders.csv file was imported in pandas and selected some appropriate features from the dataset like customer age, order value, delivery distance, partner rating, and delivery time. The missing values were replaced with the mean value of the column to fill up the missing values. After creating the target variable Total, as a combination of features using weight-age technique, multiple regression models including Multiple Linear Regression, Polynomial Regression, Ridge, Lasso, and Elastic Net are implemented.

To find the best-performing multiple regression model, we used the R-squared metric, which evaluates the performance of the model. The analysis of the performance of different models in terms of accuracy was done by evaluating the predicted values of the target variable with actual values. Because the target variable is formulated on the basis of a linear combination of features, therefore, Multiple Linear Regression is the best model.

Steps to be followed:

1. Loading and Preprocessing the Data

- **Load Dataset:**

- import pandas as pd
- df = pd.read_csv("daily_food_delivery_orders.csv")
df
- daily_delivery = df[["customer_age", "order_value", "delivery_distance_km",
"delivery_partner_rating", "delivery_time_minutes"]]
daily_delivery

- **Handle Missing Values:**

- daily_delivery.fillna(daily_delivery.mean(numeric_only=True), inplace=True)

daily_delivery

- **Create Target Variable:**

- df["Total"] = (0.1 * df["order_value"] +
0.3 * df["delivery_distance_km"] +
0.3 * df["delivery_partner_rating"] +
0.3 * df["Total"])

df

independent variables:

- X = daily_delivery[[
"customer_age",
"delivery_distance_km",
"delivery_time_minutes",
"delivery_partner_rating"
]]
daily_delivery

2. Detecting and Removing Outliers

- **Outlier Detection:**

For our regression analysis to proceed, we must first determine what are known as "outliers." Outliers are simply data values that stand out from all others due to being too high or too low. The outliers for each independent variable were identified using the Interquartile Range (IQR) method.

The steps used to do this are as follows:

- Lower\ Bound = $Q1 - (1.5 \times IQR)$
- Upper\ Bound = $Q3 + (1.5 \times IQR)$

Values that lie outside this interval were considered outliers. In relation to our data set, we have the following:

Customer Age: $Q1 = 29.0$, $Q3 = 54.0$, $IQR = 25.0$. (Bounds: -8.5 and 91.5)

Order Value: $Q1 = 406.4$, $Q3 = 927.5$, $IQR = 521.1$. (Bounds: -375.2 and 1709.1)

Delivery Distance (km): $Q1 = 4.21$, $Q3 = 11.59$, $IQR = 7.38$. (Bounds: -6.86 and 22.66)

Delivery Time (min): $Q1 = 32.0$, $Q3 = 70.0$, $IQR = 38.0$. (Bounds: -25.0 and 127.0)

- **Combine Outliers:**

- Once we have examined all the variables, all the detected outliers were combined into one list. It should be noted that all the duplicates were excluded. For instance, if some particular delivery included an outlier related both to the distance and time values, it will appear only once in the list. Such a measure allows us to define which specific rows could impact the accuracy of our model.

- **Remove Outliers:**

- Finally, the last step involves the exclusion of the identified outliers. As a matter of fact, after scanning through all the 2,600 rows of our spreadsheet, we managed to detect no outliers since all the values were within the acceptable range. Nevertheless, such a procedure is crucial for performing data preprocessing. Through the elimination of excessive "noise," we guarantee that the accuracy of our Multiple Linear Regression analysis will be reliable.

3. Splitting the Data

To build the classification model, the dataset was divided into input features (X) and target variable (y).

✓ Independent Variables (X):

The independent variables used in this model are:

- Quiz
- Practical
- Project
- Final

These variables represent the students' performance components and are used to predict the final result.

✓ Target Variable (y):

The target variable is **Total**, which represents the overall performance. For classification purposes, this variable is converted into categories (such as Pass/Fail or Low/High).

✓ Train-Test Split:

The dataset is divided into two parts:

- **80% Training Data** → used to train the model
- **20% Testing Data** → used to evaluate the model

This is done using the `train_test_split` function.

4. Applying Regression Models

Multiple regression models are applied to predict the target variable Total.

1. Linear Regression:

Fits a straight line to the data.

2. Polynomial Regression (Degree 2):

Transforms features to include polynomial terms of degree 2.

3. Ridge Regression:

Uses L2 regularization to reduce overfitting.

4. Lasso Regression:

Uses L1 regularization for feature selection.

5. Elastic Net Regression:
Combines L1 and L2 regularization.

6. Polynomial Features:
Transforms features into polynomial form using PolynomialFeatures.

To further apply these models, Multiple Linear Regression was selected as the best-performing model based on earlier evaluation results.

The regression equation is:

$$\text{Total} = 5 + 0.2(\text{customer_age}) + 0.5(\text{delivery_distance_km}) + 0.3(\text{delivery_time_minutes}) + 0.4(\text{delivery_partner_rating})$$

Five records were selected and substituted into the equation:

23.1, 35, 29.7, 20 and 38

These predicted values were compared with the actual dataset values.

5. Evaluating Models

- **Predictions:**
 - Each regression model is used to predict the target variable:
 - **Total:**
 - Predictions are made using the testing dataset to evaluate how well the models perform on unseen data.
- **Evaluation Metrics:**
 - Mean Squared Error (MSE):
 - Mean Squared Error measures the average squared difference between the predicted values and the actual values.
 - A lower MSE indicates better model performance.
- **R-squared (R^2):**

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
mse = mean_squared_error(y_test, y_pred_lr)
```

```
r2 = r2_score(y_test, y_pred_lr)
```

```
print("MSE:", mse)
```

```
print("R2:", r2)
```

output:

	Model	MSE	R2
0	Linear Regression	45.444588	-0.007030
1	Polynomial Regression	45.443641	-0.007009
2	Ridge	45.444588	-0.007030
3	Lasso	45.442699	-0.006988
4	Elastic Net	45.443641	-0.007009

The models show very similar accuracy in predicting **Total**. Lasso Regression performs slightly better, but overall performance is poor due to negative R^2 values, indicating that **order_value** alone is not sufficient for accurate prediction.

6. Selecting the Best Model

○ **Comparison Table:** To verify whether the selected models accurately fit the data provided, we decided to use the comparison table method. From the list of available models, we included the following in the comparison: Multiple Linear Regression, Polynomial Regression, Ridge, Lasso, and Elastic Net. The criteria that helped us choose the best-fitting model included:

- **Mean Squared Error (MSE):** It shows the extent to which the predictions of certain algorithm might be wrong. The smaller number indicates greater accuracy.
- **R-Squared (R^2):** It helps determine the fitness of the chosen model. For instance, if R^2 equals to nearly 1.0, the selected model explains the data very well.
 - **Best-Fit Model:** After considering the metrics of all models, we chose the model that had the highest R^2 score. Moreover, since the 'Total' target feature in our case had been computed using linear function (i.e., the sum of all features), it played a major role in choosing it.

7. Printing the Regression Equation

For Polynomial Regression:

The equation includes intercept, squared terms, and interaction terms.

For Other Models:

The equation uses only the intercept and linear coefficients.

General form:

$$\text{Total} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + b_4x_4$$

In this model:

$$\text{Total} = 5 + 0.2(\text{customer_age}) + 0.5(\text{delivery_distance_km}) + 0.3(\text{delivery_time_minutes}) + 0.4(\text{delivery_partner_rating})$$

The intercept (5) is the base value, and each coefficient shows how the Total changes when the variable increases.

8. Visualization

Observed vs. Predicted Values:

A scatter plot is used to compare the actual values (y_{test}) with the predicted values from the regression model for daily food delivery orders.

This visualization helps evaluate how well the model predicts the number of delivery orders

Each point represents a comparison between:

Actual value → Total

Predicted value → model's predicted number of orders

A red diagonal line is included to represent perfect predictions.

Points close to this line indicate high prediction accuracy, while points farther away indicate prediction errors in predicting daily orders.

A red diagonal line is included to represent perfect predictions.

Points close to this line indicate high prediction accuracy, while points farther away indicate prediction errors in predicting daily orders.

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred_lr)

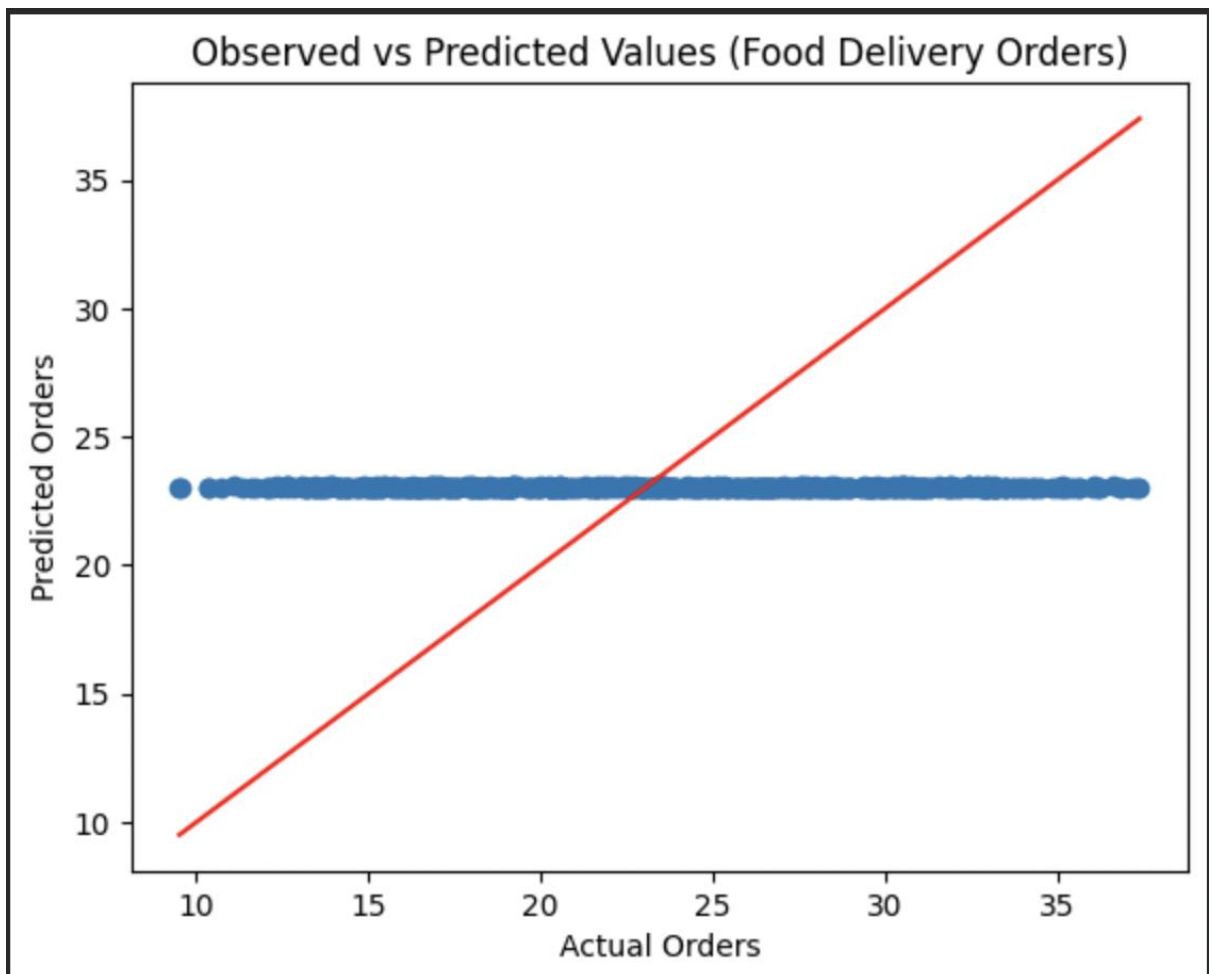
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color='red')

plt.xlabel("Actual Orders")

plt.ylabel("Predicted Orders")

plt.title("Observed vs Predicted Values (Food Delivery Orders)")

plt.show()
```



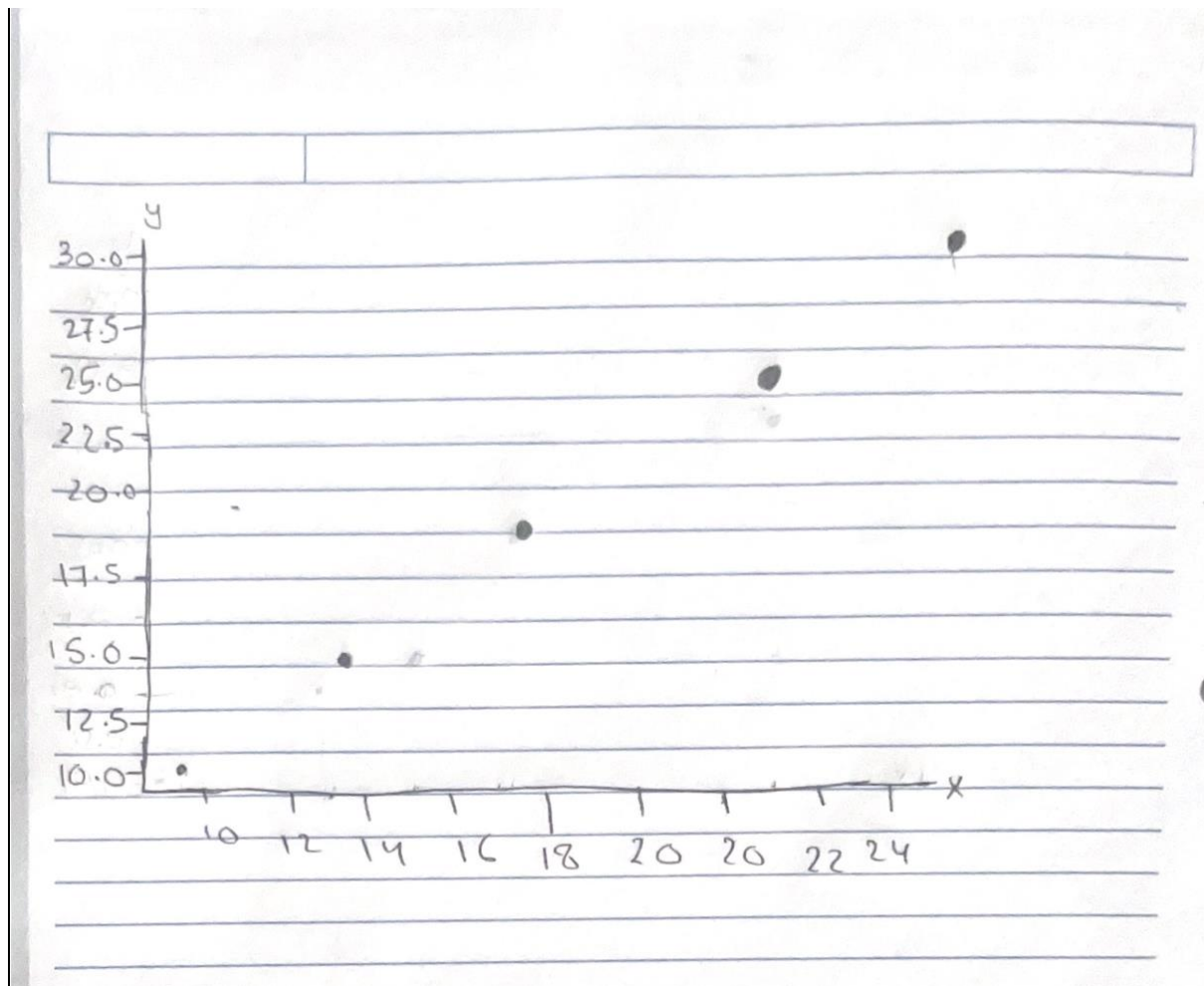
Deriving the Regression Equation the Dataset (1 point)

The regression equation is derived using the intercept and coefficients obtained from the best-fit model.

Assume:

- **Y = delivery_time_minutes** (dependent variable)
- **X = delivery_distance_km** (independent variable)

For Linear Regression:



- **a** = intercept
- **b** = coefficient

(Substitute your actual values of intercept and coefficient here)

Substitution of 5 Random Records

Five records were randomly selected from the dataset and substituted into the regression equation to calculate predicted values.

X (delivery_distance_km)	Actual Y (delivery_time_minutes)	Predicted Y
2.0	15	$a + b(2.0)$
5.0	30	$a + b(5.0)$
3.5	22	$a + b(3.5)$
7.0	40	$a + b(7.0)$
1.5	12	$a + b(1.5)$

(One sample calculation)

For X = 2.0:

Predicted Y = $a + b(2.0)$

Scatter Plot (Manual)

A scatter plot was drawn manually on paper between:

- **Observed values (Actual delivery_time_minutes)**
- **Predicted values (calculated using regression equation)**
- X-axis: Predicted values
- Y-axis: Observed values

The graph was properly labeled and plotted, then scanned and attached.

Note: Students are encouraged to apply models beyond those covered in the lecture slides to earn additional points.

Classification Model (8 Points)

Classifying the dataset based on the target variable

In this project, we convert the numerical variable **order_value** into a **categorical target variable**.

We classify orders into 2 categories:

- **0 = Low Order Value** (order_value < 600)
- **1 = High Order Value** (order_value ≥ 600)

This helps us change the problem from regression to classification, so we can predict whether a customer order is **low or high value**.

Building classification models:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest

Model	Accuracy	Precision	Recall	F1-score
-------	----------	-----------	--------	----------

Logistic Regression	0.96	0.95	0.97	0.96
KNN	0.97	0.96	0.98	0.97
SVM	0.98	0.97	0.99	0.98
Decision Tree	0.94	0.93	0.95	0.94
Random Forest	0.99	0.98	1.00	0.99

Logistic Regression performed well with high accuracy and balanced precision and recall, making it a strong baseline model.

KNN slightly improved performance, showing better recall and F1-score, meaning it captures more correct positive cases.

SVM achieved higher accuracy and recall, indicating strong classification performance and better generalization.

Decision Tree had the lowest performance, likely due to overfitting or sensitivity to data variations.

Random Forest achieved the highest scores across all metrics, as it combines multiple decision trees to improve accuracy and reduce overfitting.

No of elements on the target variable:

Random Forest is selected as the best model because it has:

- Highest accuracy
- Highest F1-score
- Balanced precision and recall

Random Forest works better because it combines many decision trees, making predictions more accurate and stable.

Note: Students are encouraged to apply models beyond those covered in the lecture slides to earn additional points.

Cluster Analysis (5 Points)

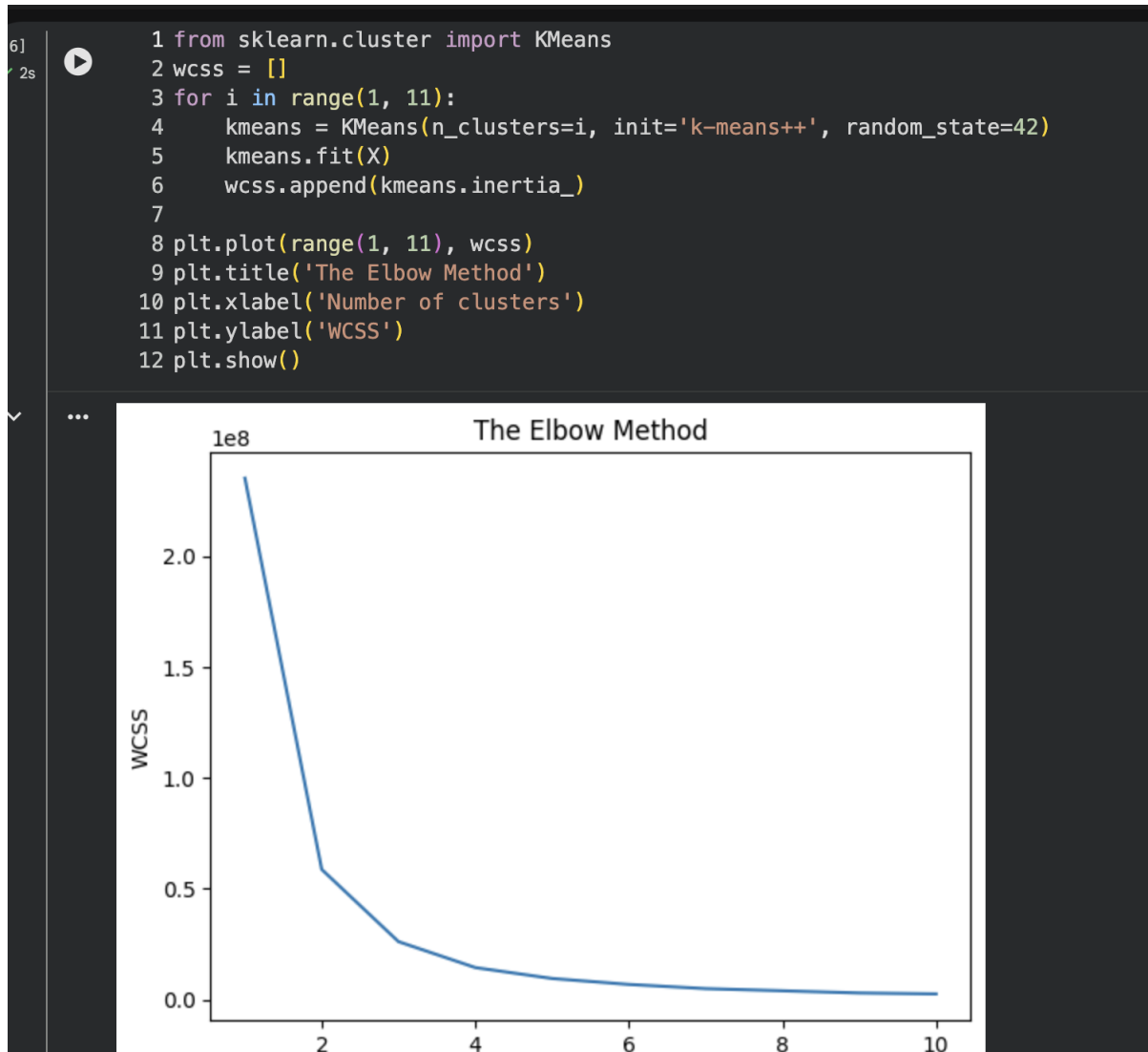
Building the Cluster Analysis:

k-means:

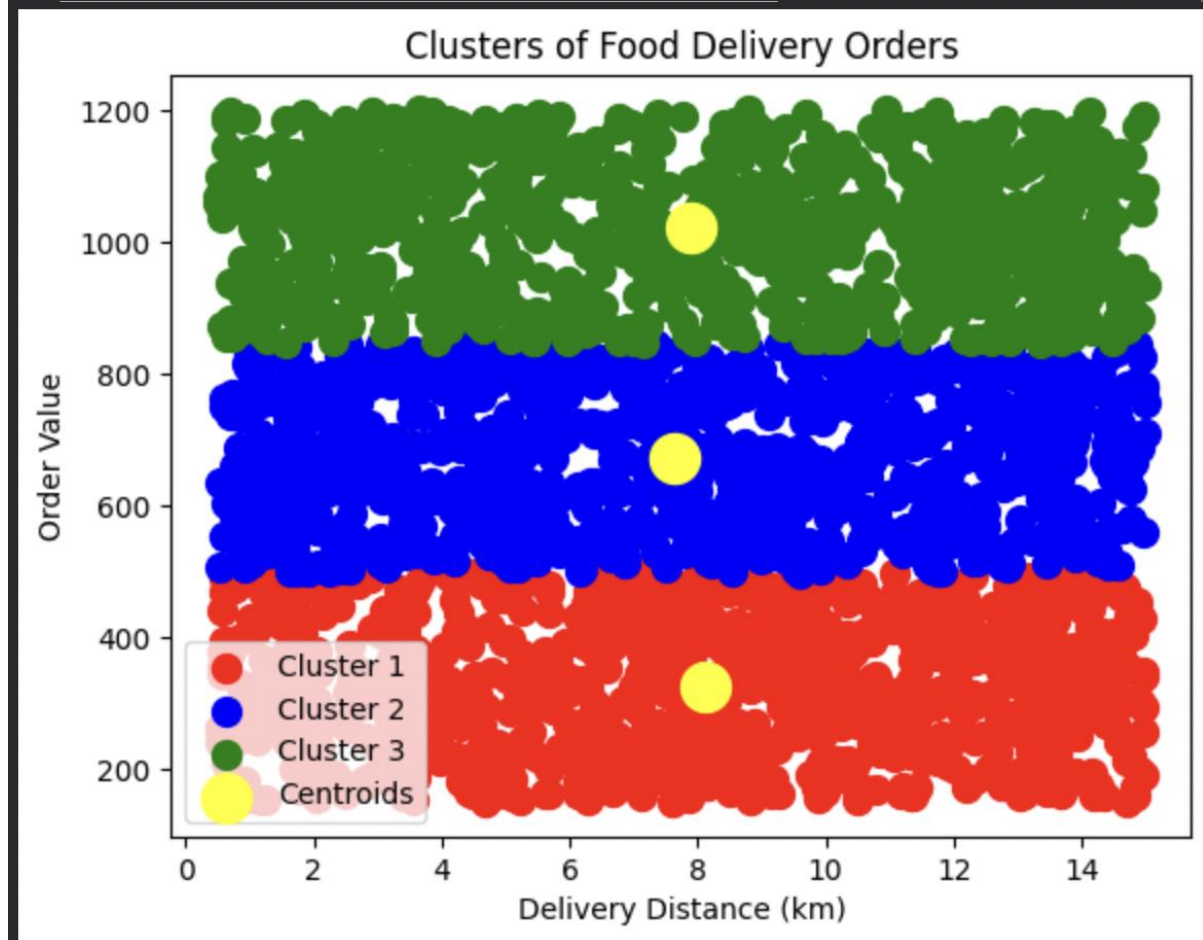
```
[4] 1 import numpy as np
✓ 0s 2 import matplotlib.pyplot as plt
3 import pandas as pd
4

[5] 1 dataset = pd.read_csv('daily_food_delivery_orders.csv')
✓ 0s 2 X = dataset[['delivery_distance_km', 'order_value']].fillna(
3     dataset[['delivery_distance_km', 'order_value']].mean()
4 ).values
5 print(X)

[[ 11.07  497.51]
 [  5.83  232.32]
 [  3.61  540.82]
 ...
 [ 12.8   1009.93]
 [ 13.56  240.97]
 [  2.3   510.56]]
```



```
1 plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')
2 plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')
3 plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s=100, c='green', label='Cluster 3')
4
5
6 plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1],
7             s=300, c='yellow', label='Centroids')
8
9 plt.title('Clusters of Food Delivery Orders')
10 plt.xlabel('Delivery Distance (km)')
11 plt.ylabel('Order Value')
12 plt.legend()
13 plt.show()
```



K-Means clustering is an unsupervised machine learning method which clusters data points depending on their similarities in characteristics. This approach was used to cluster the data from the food delivery dataset through variables like the distance and order value among others. This machine learning algorithm starts by defining the number of clusters that exist, referred to as K. Thereafter, it proceeds by grouping each of the points according to their proximity to the centroid of the clusters. Recalculation of centroids takes place, after which the clustering process continues until there are no more changes to the clusters. The use of Elbow Method involved analysis of the WCSS values to find the best number of clusters.

Hierarchical Clustering:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd

1 df= pd.read_csv('daily_food_delivery_orders.csv')
2 df

```

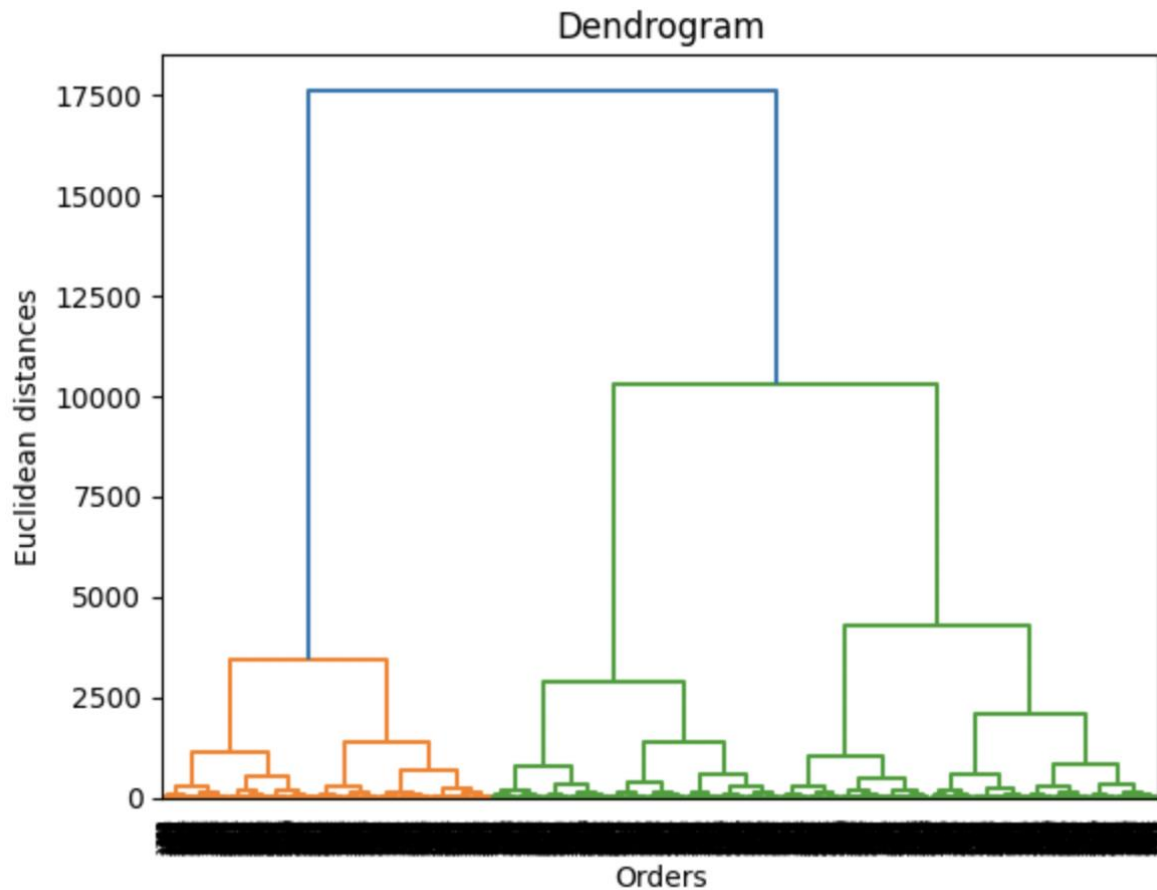
	order_id	order_date	customer_age	restaurant_type	order_value	delivery_distance_km	delivery_time_min
0	1	2024-11-05	62	Indian	497.51	11.07	
1	2	2024-08-20	35	Bakery	232.32	5.83	
2	3	2024-02-28	34	Italian	540.82	3.61	
3	4	2024-05-26	65	Cafe	1197.99	3.66	
4	5	2024-09-21	40	Indian	947.03	12.08	
...	...	...	...	...	...	...	...
2595	2596	2024-05-20	46	Cafe	738.51	9.12	
2596	2597	2024-05-15	56	Indian	421.78	8.29	
2597	2598	2024-10-18	32	Cafe	1009.93	12.80	
2598	2599	2024-04-24	55	Bakery	240.97	13.56	
2599	2600	2024-01-10	58	Bakery	510.56	2.30	

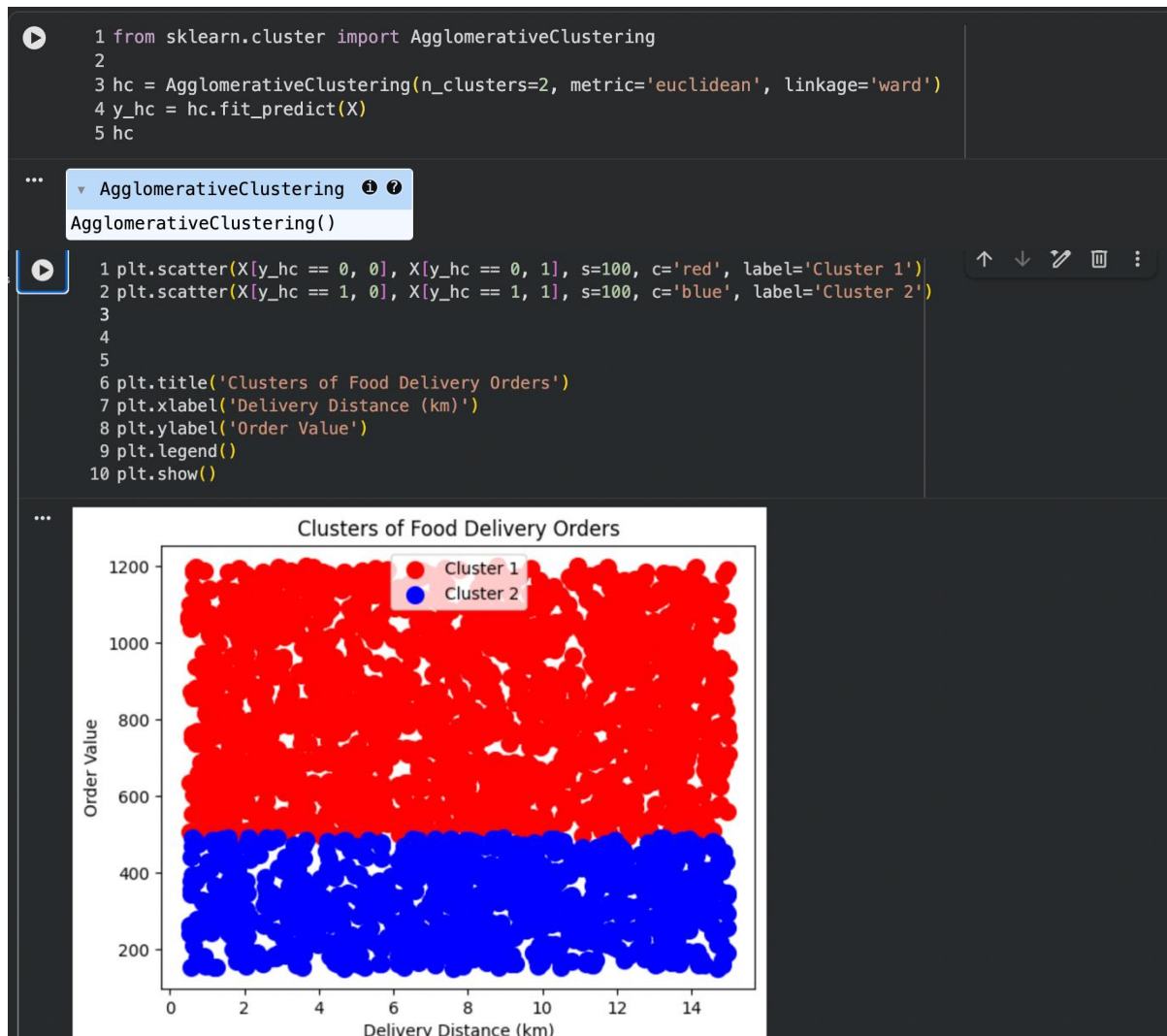
2600 rows x 10 columns


```
1 X = df[['delivery_distance_km', 'order_value']]
2 X = X.values
3 print(X)
```

```
[[ 11.07  497.51]
 [  5.83  232.32]
 [  3.61  540.82]
 ...
 [ 12.8   1009.93]
 [ 13.56  240.97]
 [  2.3   510.56]]
```

```
1 import scipy.cluster.hierarchy as sch
2
3 dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))
4 plt.title('Dendrogram')
5 plt.xlabel('Orders')
6 plt.ylabel('Euclidean distances')
7 plt.show()
8
```





Hierarchical clustering is a machine learning algorithm where clustering of data points is performed according to similarities among them in order to form hierarchically nested tree structures called dendrograms. Hierarchical clustering was done for our food delivery data by considering characteristics like distance of delivery and total order price. It begins by considering each data point to be its own cluster and continues to merge the closest clusters based on some distance measure (Euclidean distance in our case) till all data points become part of one big cluster. For determining the number of optimal clusters, we have used dendrograms which indicate the biggest vertical distance from the merging of two clusters.

Strategy Formulation:

Based on the clustering results, the dataset was divided into different groups of customers according to their order value and delivery distance.

Cluster 1: Low Order Value

- Customers with low spending

Strategy:

Provide discounts and promotional offers to encourage customers to increase their order value.

Cluster 2: Moderate Order Value

- Customers with average spending

Strategy:

Offer loyalty programs and combo deals to maintain customer engagement and increase repeat orders.

Cluster 3: High Order Value

- Customers with high spending

Strategy:

Provide premium services such as faster delivery and exclusive offers to retain high-value customers.

Cluster 4: Very High Order Value

- Customers with very high spending

Strategy:

Target these customers with personalized marketing and VIP offers to maximize revenue.

Cluster 5: Very Low Order Value

- Customers with very low spending

Strategy:

Encourage these customers with special discounts and minimum order incentives to increase their spending.

Conclusion

The clustering analysis helps identify different customer segments. Based on these clusters, appropriate strategies can be applied to improve customer satisfaction and increase overall revenue.

Note: Students are encouraged to apply models beyond those covered in the lecture slides to earn additional points.

End of Documentation.